

Subqueries en FROM y SELECT

En la clase anterior descubrimos que una subquery en el WHERE funciona como un filtro dinámico: calcula algo primero y usa ese resultado para decidir qué filas incluir. Pero el WHERE no es el único lugar donde puedes poner una subquery. En esta clase la movemos a dos posiciones completamente distintas — el FROM y el SELECT — y vas a ver que la misma sintaxis (una query dentro de paréntesis) se comporta de manera muy diferente según dónde la coloques.

La forma más fácil de recordarlo es así: en el WHERE, la subquery es un filtro. En el FROM, es una tabla virtual. Y en el SELECT, es un valor calculado que se genera fila por fila. Con esta clase cerramos ese triángulo.

Subquery en el FROM: convertir una consulta en tabla

Cuando pones una subquery dentro del FROM, SQL Server la ejecuta primero y trata el resultado como si fuera una tabla más. Una tabla que no existe físicamente en la base de datos, pero que para efectos de tu consulta se comporta exactamente como una. A esto se le llama **tabla derivada**.

¿Para qué sirve? Para trabajar en dos etapas. Primero haces un cálculo — agrupas, filtras, transformas — y después operas sobre ese resultado como si fuera una tabla normal: la ordenas, le sacas un TOP, la cruzas con otra tabla, lo que necesites.

Veámoslo con un caso de TiendaLatam. La pregunta es: *"¿Cuáles son los 3 países con mayor ticket promedio por pedido?"*. Para responder esto necesitas primero calcular el ticket promedio por país (esa es la primera etapa), y después ordenar ese resultado y quedarte con los tres más altos (esa es la segunda etapa). La subquery en el FROM hace la primera etapa, le pones un nombre — digamos "resumen" — y la query principal trabaja sobre ese "resumen" como si fuera cualquier otra tabla.

Un detalle técnico que no puedes olvidar: en SQL Server, toda tabla derivada necesita un alias. Es decir, ese nombre que le pones después del paréntesis (el AS resumen) es obligatorio. Si no lo pones, la consulta no corre.

Ahora, ¿cuándo conviene usar una tabla derivada en vez de resolver todo en una sola consulta con JOINS? La respuesta corta es: cuando el cálculo intermedio es complejo. Si necesitas hacer un GROUP BY con varias agregaciones y después filtrar u ordenar sobre esos resultados, meterlo todo en una sola pasada se vuelve difícil

de leer y a veces ni siquiera es posible. La tabla derivada te permite separar la lógica en capas, y eso hace que la consulta sea más clara y más fácil de mantener.

En la clase vimos un segundo ejemplo donde la subquery calcula, para cada país, la cantidad de clientes activos, las ventas totales y las ventas por cliente — un cálculo bastante más pesado. Todo eso queda encapsulado en la subquery con el alias "estadísticas", y la query principal simplemente pide el TOP 5 ordenado por ventas por cliente. Cada pieza tiene su responsabilidad clara.

Subquery en el SELECT: un valor calculado por cada fila

Mover la subquery al SELECT es un cambio de propósito importante. Acá ya no estás creando una tabla ni filtrando filas. Lo que estás haciendo es agregar **una columna calculada** al resultado, donde el valor de esa columna se obtiene ejecutando la subquery para cada fila que devuelve la query principal.

El caso que trabajamos en clase es un clásico: *"Muéstrame cada país con su porcentaje del total de ventas global"*. Para calcular un porcentaje necesitas dos cosas: las ventas del país (que vienen del GROUP BY de la query principal) y las ventas totales de todos los países (que es un número global). Ese número global lo calculas con una subquery escalar directamente en el SELECT.

El resultado es una tabla donde cada fila tiene el nombre del país, sus ventas, las ventas globales y el porcentaje. La subquery se encarga de traer ese total global y ponerlo al lado de cada país para que la división funcione.

Hasta acá todo bien. Pero hay un tema de rendimiento que necesitas tener en la cabeza. Cuando la subquery está en el SELECT, se ejecuta **una vez por cada fila** del resultado. Si tu consulta devuelve 12 filas, la subquery corre 12 veces. Con 12 filas no pasa nada, pero si mañana tu consulta devuelve 10,000 filas, esa subquery se ejecuta 10,000 veces. Y si la subquery es costosa — por ejemplo, si tiene sus propios JOINS y filtros — el rendimiento se puede degradar rápido.

La alternativa elegante: declarar variables

Hay una forma de evitar ese problema de rendimiento que además hace tu código más limpio. Si el valor que calcula la subquery es siempre el mismo — como el total global de ventas, que no cambia de fila en fila — puedes calcularlo una sola vez, guardarlo en una variable, y después usar esa variable en tu consulta.

En SQL Server se hace con **DECLARE**. Declaras una variable (por ejemplo **@TotalGlobal**), le asignas el resultado de la subquery con un **SELECT**, y listo: ese valor queda en memoria. Después, en tu query principal, donde antes tenías la subquery entre paréntesis, simplemente pones **@TotalGlobal**. El resultado es exactamente el mismo, pero en vez de ejecutar la subquery una vez por cada fila, la ejecutaste una sola vez al principio.

Es más eficiente y, honestamente, más fácil de leer. Cuando alguien mira tu consulta y ve **@TotalGlobal** en una fórmula, entiende inmediatamente qué representa ese valor. Cuando ve una subquery de cuatro líneas metida dentro de un cálculo matemático dentro del **SELECT**, tiene que detenerse a descifrar qué hace.

Dicho esto, las variables no siempre son la solución. Funcionan bien cuando el valor es constante para toda la consulta (un total global, un promedio general, una fecha de corte). Pero si el valor de la subquery cambia según la fila — por ejemplo, si necesitas el total de ventas del país de cada cliente — ahí sí necesitas la subquery en el **SELECT** o resolver el problema con un **JOIN**.

El triángulo completo

Con esta clase ya tienes el mapa completo de dónde pueden vivir las subqueries. En el **WHERE**, funcionan como filtros dinámicos: calculan un valor o una lista y deciden qué filas entran en el resultado. En el **FROM**, se convierten en tablas derivadas: ejecutan un cálculo complejo y te dejan trabajar sobre ese resultado en una segunda etapa. Y en el **SELECT**, agregan valores calculados fila por fila, aunque con la advertencia de que pueden ser costosas si el resultado tiene muchas filas.

En la próxima clase vamos a llegar a los CTEs, que son — según el instructor — la forma más profesional de escribir consultas complejas con múltiples subqueries. Pero para apreciar por qué los CTEs son tan útiles, primero necesitabas entender bien las subqueries en sus tres posiciones. Eso ya lo tienes.

Desafío de cierre

Calcula el porcentaje de ventas de cada categoría de producto sobre el total global, usando una subquery escalar en el **SELECT**. Después, reescribe esa misma consulta declarando una variable con **DECLARE** para almacenar el total global. Compara ambas versiones y pregúntate: ¿cuál te resulta más legible? ¿Cuál crees que es más eficiente y por qué? Comparte las dos versiones en los comentarios de la clase.